# AEGIS — Autonomous Evidence-Gated Intake & Governance System

CIPHER 2.0 · Scenario 03 · Phase 2 Prototype · Team Amateurs, Informatics Institute of Technology

Evidence-weighted capstone team allocation with auditable governance **ALL DATA SYNTHETIC**

## 1 Problem & Solution

**The problem.** Allocating students to capstone projects and teams is, in most institutions, a manual and opinion-driven exercise. Skills are taken from self-declaration at face value — so a confident over-claimer outranks a quiet expert; near-duplicate project topics slip past tired reviewers; and disengagement or free-riding only surfaces at the final viva, when it is too late to intervene. None of it leaves an auditable trail, so allocation decisions cannot be defended when challenged.

**What AEGIS does.** AEGIS is an evidence-gated pipeline — `intake → de-duplicate → match → form → monitor` — that treats every self-declared skill as a *claim to be weighted by its evidence*, not a fact. Unproven claims are discounted, near-duplicate projects are blocked for review before allocation, teams are formed to lift the weakest team rather than the average, and engagement is monitored continuously so faculty are alerted to ghosting, sympathy-carrying and burnout while there is still time to act.

**What it is.** A pure Python decision engine driving a Next.js dashboard over a Supabase / Postgres database, with object-scoped Row-Level Security and an append-only, hash-chained audit log as the governance substrate. Every state-changing action is recorded; every allocation is explainable.

## 2 The Algorithm

### 2.1 Evidence weighting — the Dunning–Kruger correction

Each declared skill is adjusted by a confidence factor reflecting the strength of its evidence:

$$\hat{A} = L \times C$$

where **L** is the declared level (1–5) and **C** is the confidence factor keyed to the evidence basis:

| Evidence basis | C | Meaning |
|---|---|---|
| verified | 1.0 | institutional grade / assessed record |
| portfolio | 0.8 | portfolio or prior work submitted |
| self-report | 0.6 | self-report only (default; no LMS link) |
| contradicted | 0.5 | self-report contradicted by observed throughput |

The **contradicted** tier is the Dunning–Kruger correction: when a high self-claim conflicts with measured task throughput, confidence is halved.

**Live case — STU_08.** Declares Technical **5.0** with no supporting evidence, contradicted by throughput → C = 0.5 → $\hat{A} = 5.0 \times 0.5 = 2.5$. The inflated claim is corrected before it can distort team formation; STU_08 ultimately lands in the critical team P_05.

### 2.2 Duplicate detection — TF-IDF cosine gate

Project abstracts are vectorised with TF-IDF and compared by cosine similarity. Any pair scoring **≥ 0.75** is flagged and blocked for supervisor review before allocation proceeds.

**Live case.** Projects **P_02** and **P_03** flagged at **0.96** — well above the 0.75 gate — and held for review rather than silently allocated as two distinct projects.

### 2.3 Matching — Student–Project Allocation (SPA)

Allocation uses the Abraham–Manlove Student–Project Allocation algorithm, honouring students' ranked project preferences against supervisor and project-capacity constraints — a stable matching rather than first-come allocation.

### 2.4 Formation — maximin floor-lifting

Teams are formed to maximise the capability of the *weakest* team (a maximin objective), rather than the average — deliberately lifting the floor so no team is left non-viable. The resulting health-band spread on the live cohort:

| Team | Health score | Band |
|---|---|---|
| P_04 | 84 | **Healthy** |
| P_01 | 69 | At-Risk — carry imbalance + burnout |
| P_05 | 41 | **Critical** — ghost + overload + low completion |

### 2.5 Monitoring — four engagement signals

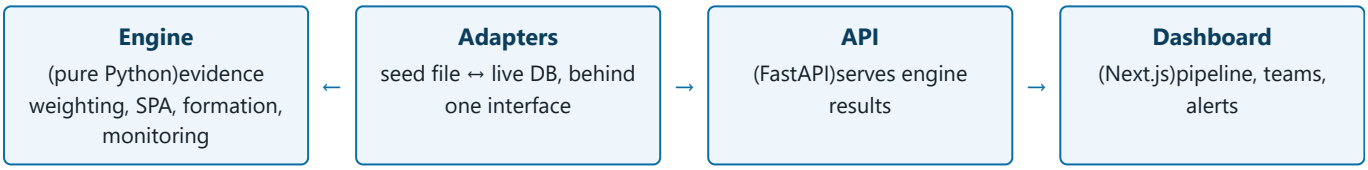
Over a 14-day sprint window the engine raises four governed signals:

- **Ghosting (Tier-3):** 10+ consecutive zero-input days → faculty critical alert.
- **Sympathy-carry:** one member authoring ≥ 0.95 of another's assigned tasks.
- **Burnout:** individual utilisation ≥ 2× the team average.
- **Health-band alerts:** team score crossing into At-Risk or Critical bands.

> **Live cases. STU_07** (team P_05) authors **zero events across all 14 sim-days** — a pure ghost, exceeding the 10-day Tier-3 threshold. In team P_01, **STU_01** authors **≥ 0.95** of STU_05's assigned tasks (sympathy-carry), and STU_01's resulting 10-event load trips the **burnout** tripwire — carrier and carried surfaced together.

## 3 Architecture

AEGIS is layered so that decision logic is isolated from data sources and transport:




- **Engine purity.** The engine imports *nothing* from the adapter layer — it operates on plain domain objects, so it is unit-testable on fixed data with no database in the loop.
- **Adapter env-switch.** A single environment switch flips the data source between the synthetic seed and the live Supabase database *without any change to engine code* — the same logic runs in demo and production.
- **Governance substrate.** Supabase Postgres with Row-Level Security enforces who can read which rows; the API never returns data the caller is not entitled to.
- **Drive integration.** Team workspace provisioning runs on OAuth-as-user with least-privilege scopes (see §4).

## 4 Security & Governance

Controls are framed against the standards actually applied, with evidence rather than assertion.

- **OWASP API1:2023 (BOLA) remediation.** The `team_monitoring` surface was hardened from authentication-only to **object-scoped RLS**: an authenticated user can no longer read another team's monitoring rows — access is checked against row ownership, not merely a valid session.
- **Three-tier RLS verification.** Verification distinguishes *toggle-exists* ≠ *policy-exists* ≠ **policy-enforces**. RLS being "on" is necessary but not sufficient; enforcement is confirmed at the row level, not at the toggle.
- **Privilege-escalation controls.** The `audit_log` is **append-only and hash-chained** (each entry binds the previous, so tampering is detectable). User **role is hard-forced server-side** — a client cannot assign itself a privileged role.
- **Least-privilege Drive.** The integration requests only `drive.file` + `drive.activity.readonly`, acting as the user via OAuth. The app can touch only the files it itself created — never the user's wider Drive.
- **CIS Control 3 (data protection).** No credential material exists anywhere in git history (verified by a targeted full-history pattern sweep for JWT/PEM/OAuth-secret/DB-URL signatures — see `PUBLICATION_CHECKLIST.md`). Seed data is synthetic only, using **RFC 2606 reserved `@aegis.test`** addresses that cannot resolve to a real mailbox.

## 5 Live Results

Running the full pipeline over the **70-student cohort** produces **15 teams + 1 exception pool**. Engineered probe students *within that cohort* each demonstrate one mechanism end-to-end:

| Mechanism | Live evidence | Outcome |
|---|---|---|
| Evidence weighting | STU_08: declared 5.0, contradicted → Â = 5.0 × 0.5 = 2.5 | Inflated claim corrected (amber) |
| Duplicate detection | P_02 / P_03 cosine = 0.96 (≥ 0.75 gate) | Blocked for review |
| Ghosting (Tier-3) | STU_07: 0 events / 14 sim-days | Faculty critical alert |
| Sympathy-carry + burnout | STU_01 authors ≥ 0.95 of STU_05's tasks; 10-event load | Carry + burnout alerts |
| Formation spread | P_04 = 84 / P_01 = 69 / P_05 = 41 | Healthy / At-Risk / Critical bands |

**[ FIG 1 ]**

Pipeline stepper — live dashboard (intake → de-dup → match → form → monitor)

*drop screenshot here · captured from the live-data dashboard*

**[ FIG 2 ]**

Team health bands — P_04 Healthy 84, P_01 At-Risk 69, P_05 Critical 41

*drop screenshot here · captured from the live-data dashboard*

**[ FIG 3 ]**

Alert inbox — STU_07 ghost (Tier-3) + STU_01/STU_05 carry & burnout

*drop screenshot here · captured from the live-data dashboard*

## 6 Limitations & Scaling

### Documented, not built (deliberate prototype scope)

- **LMS grade sync** (the verified-evidence T1 channel) is mocked; with no institutional API the system defaults to C = 0.6 self-report and works without it.
- **Server-side caching / job queue** (e.g. Redis, BullMQ) for high-volume alert dispatch — noted as a scale-up path, not built at prototype scale.
- **Off-peak batch re-similarity sweeps** across the full cohort, versus the current per-submission duplicate check.

### Honest engineering caveats

- **Non-atomic migrations.** Schema migrations are applied in manual order with no down-migrations; a failed mid-sequence apply needs manual reconciliation.
- **Schema-drift risk.** The synthetic seed shape and the live DB shape are maintained in parallel; they can drift if a migration lands on one path but not the other.

- **Drive ownership constraint.** On consumer Gmail a service account cannot own files (zero storage quota), so provisioning uses OAuth-as-user. A Google Workspace **Shared Drive** is the institution-grade path once that access exists.

## 7 Data Statement

**All personal data in this system is synthetic and was generated for demonstration; it corresponds to no real individual.**